Light

Boolean Logic & Truth Tables

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Logical operators
2. Truth tables
3. Applying logic
Worked examples
Quick reference
Typical pitfalls

Edexcel GCSE Computer Science — **Boolean Logic (1CP2/01 Topic 1: Computational Thinking)**. This topic covers the three logical operators (AND, OR, NOT) and how to construct and complete truth tables with up to three inputs to solve problems.
Exam questions typically ask you to **complete** truth tables, **identify** the output of logical expressions, and **apply** Boolean logic to programming scenarios. Expect questions worth 2—4 marks requiring you to work through logical operations systematically.

Why Boolean logic matters

Boolean logic is the foundation of how computers make decisions. Everything in a computer ultimately comes down to true/false decisions represented by 1s and 0s.

- **Selection statements:** Every `if` statement uses Boolean logic to decide which code to execute
- **Loop conditions:** `while` loops continue or stop based on Boolean conditions
- **Validation:** Checking multiple conditions (e.g., password AND username correct)
- **Search filters:** Combining criteria (e.g., red AND size large OR blue)
- **Paper 1 Topic 1:** Truth tables appear regularly as 2—4 mark questions

What you must know

This checklist covers the key exam-visible concepts from Edexcel 1CP2 Topic 1: Computational Thinking (specifically 1.2.3 and 1.3.1).

- **Understand** how the three logical operators (AND, OR, NOT) work
- **Apply** logical operators in algorithms and programming
- **Construct** truth tables with one, two, or three inputs
- **Complete** truth tables showing all possible input combinations
- **Determine** the output of Boolean expressions for given inputs
- **Use** truth tables to solve problems and verify logic

> **Specification link:** This topic covers Edexcel 1CP2 Paper 1 Topic 1: Computational Thinking — specifically 1.2.3 (logical operators AND, OR, NOT in algorithms) and 1.3.1 (truth tables with up to three inputs).

Toolkit

These are the key patterns and reference tables you need for Boolean logic questions. Memorise the three operator truth tables — they are the building blocks for everything else.

The three logical operators

AND

Output is TRUE only when **both** inputs are TRUE.

Think: "Both must be true"

A	B	A AND B
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

OR

Output is TRUE when **at least one** input is TRUE.

Think: "One or more true"

A	B	A OR B
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

NOT

Output is the **opposite** of the input.

Think: "Flip/invert it"

A	NOT A
0	**1**
1	**0**

Quick reference for counting rows

Number of inputs	Calculation	Number of rows	Input combinations
1 input	2^1	2 rows	0, 1
2 inputs	2^2	4 rows	00, 01, 10, 11
3 inputs	2^3	8 rows	000, 001, 010, 011, 100, 101, 110, 111

Python logical operators

Logic term	Python keyword	Example	Result when age=17, hasID=True
AND	`and`	`age >= 18 and hasID`	False (17 is not >= 18)
OR	`or`	`age >= 18 or hasID`	True (hasID is True)
NOT	`not`	`not hasID`	False (opposite of True)

Exam tip: In truth tables, 1 = TRUE and 0 = FALSE. Some questions use T/F instead of 1/0 — both are acceptable. In Python code, use `True` and `False` (with capital letters).

1. Logical operators

A **logical operator** (also called a Boolean operator) takes one or more Boolean inputs and produces a single Boolean output. At GCSE, you need to know three operators: AND, OR, and NOT. These operators are fundamental to how computers make decisions and process information.

Key concept: Boolean values can only be TRUE (1) or FALSE (0). There are no other possibilities. This binary nature is why Boolean logic is perfectly suited to computers, which operate using binary (0s and 1s).

1.1 The AND operator

The AND operator returns TRUE only when **all** inputs are TRUE. If any input is FALSE, the output is FALSE. This is the most restrictive operator because it requires every condition to be met.

A AND B = 1

Only when A = 1 **and** B = 1

How to remember AND: Think of AND as a strict gatekeeper. Every single condition must be satisfied before you're allowed through. Even one FALSE input blocks everything.

Real-world examples of AND:

- ATM: valid card AND correct PIN — both needed
- Login: correct username AND correct password
- Driving: old enough AND passed test AND have licence
- Exam: answered AND answer is correct

Programming example: A login system checks username AND password:

```
if (username == "admin") and (password == "secret123"): print("Access granted") else: print("Access denied")
```

Access is only granted when both the username is correct AND the password is correct. If either is wrong, access is denied.

1.2 The OR operator

The OR operator returns TRUE when **at least one** input is TRUE. It only returns FALSE when all inputs are FALSE. This is the most permissive operator.

A OR B = 1

When A = 1 **or** B = 1 **or both**

How to remember OR: Think of OR as "at least one". As long as any single condition is met, the result is TRUE. You only get FALSE when nothing works.

Important: In computer science, OR is **inclusive** — if both inputs are TRUE, the output is still TRUE. This is different from how we sometimes use "or" in everyday language.

Real-world examples of OR:

- Discount: student OR senior citizen — either qualifies
- Entry: member OR guest pass — either works
- Payment: cash OR card OR voucher — any method accepted
- Alarm: motion detected OR window opened — either triggers

Programming example: A game allows replay if the player presses 'Y' or 'y':

```
if (choice == "Y") or (choice == "y"): print("Starting new game") else: print("Thanks for playing!")
```

1.3 The NOT operator

The NOT operator **inverts** a single input. TRUE becomes FALSE, and FALSE becomes TRUE. Unlike AND and OR, NOT only takes one input.

NOT A

Returns the **opposite** of A

How to remember NOT: Simply flip the value. Whatever it is, make it the opposite.
Real-world examples of NOT:

- Door: if NOT locked, you can enter
- Game: continue while NOT game over
- Light: if NOT dark, don't turn on lights
- Queue: if NOT empty, serve next customer

Programming example: A loop continues while the game is NOT over:

```
gameOver = False while not gameOver: # game code runs here if lives == 0:
gameOver = True
```

The loop continues as long as `not gameOver` is TRUE (i.e., while gameOver is FALSE).

1.4 Combining operators

Logical operators can be combined to create more complex conditions. When combining operators, evaluate in this order (known as **operator precedence**):

1. **Brackets** — always evaluate contents of brackets first
2. **NOT** — apply NOT operations next
3. **AND** — then evaluate AND operations
4. **OR** — finally evaluate OR operations

Example: For the expression `A AND B OR C`:

- First evaluate A AND B
- Then OR the result with C

Using brackets makes the order clearer: `(A AND B) OR C`
Different meanings with brackets:

- `(A AND B) OR C` — A and B must both be true, OR C alone is true
- `A AND (B OR C)` — A must be true, AND either B or C must be true

> **Memory tip:** AND is like multiplication (both must be 1 to get 1), OR is like addition (either can give a non-zero result). This helps remember the order: multiply before add, AND before OR.

Worked Example 1 – Identifying AND output

Question: A security system grants access when both a valid keycard AND a correct PIN are provided. What is the output when keycard = TRUE and PIN = FALSE? [1 mark]

Model answer:
Output = FALSE / 0 **(1)**
Reasoning: AND requires both inputs to be TRUE. Since PIN = FALSE, the output must be FALSE.

Exam tip: For AND, if you see any FALSE input, the answer is FALSE — you don't need to check the other inputs.

Worked Example 2 – Identifying OR output

Question: A discount is applied if the customer is a member OR has a voucher. What is the output when member = FALSE and voucher = TRUE? [1 mark]

Model answer:
Output = TRUE / 1 **(1)**
Reasoning: OR requires at least one input to be TRUE. Since voucher = TRUE, the output is TRUE.

Exam tip: For OR, if you see any TRUE input, the answer is TRUE — you don't need to check the other inputs.

Worked Example 3 – Applying NOT

Question: A program continues while the variable `finished` is NOT TRUE. If finished = FALSE, does the program continue? [1 mark]

Model answer:
Yes, the program continues **(1)**
Reasoning: NOT FALSE = TRUE. The condition "NOT finished" evaluates to TRUE, so the program continues.

Exam tip: NOT simply flips the value. NOT TRUE = FALSE. NOT FALSE = TRUE.

Worked Example 4 – Evaluating combined expression

Question: Evaluate the expression `(A AND B) OR C` when A = 1, B = 0, C = 1. [2 marks]

Model answer:
Step 1: Evaluate brackets first: A AND B = 1 AND 0 = 0 **(1)**
Step 2: Evaluate OR: 0 OR C = 0 OR 1 = 1 **(1)**
Final answer: 1 (TRUE)

> **Exam tip:** Always show your working step by step. Evaluate brackets first, then NOT, then AND, then OR.

2. Truth tables

A **truth table** shows all possible combinations of inputs and the corresponding output for a logical expression. Truth tables are used to verify logic and understand how expressions behave for all possible inputs. They are a systematic way to check that your logic is correct.

> **Why truth tables matter:** Truth tables prove that logic works correctly in ALL cases, not just the ones you've tested. This is important in programming because untested edge cases often cause bugs.

2.1 Structure of a truth table

A truth table has:

- **Input columns:** One column for each input variable (A, B, C, etc.)
- **Output column(s):** The result of the logical expression
- **Rows:** One row for each possible combination of inputs

The number of rows depends on the number of inputs. Since each input can be either 0 or 1 (two possibilities), the total combinations follow powers of 2:

- 1 input → 2^1 = 2 rows
- 2 inputs → 2^2 = 4 rows
- 3 inputs → 2^3 = 8 rows

At GCSE, you only need to work with up to 3 inputs (8 rows maximum).

2.2 Writing input combinations systematically

Input combinations should be written systematically, typically in binary counting order. This ensures you don't miss any combinations.

Method for 3 inputs (A, B, C):

- The **rightmost column (C)** alternates every row: 0,1,0,1,0,1,0,1
- The **middle column (B)** alternates every 2 rows: 0,0,1,1,0,0,1,1
- The **leftmost column (A)** alternates every 4 rows: 0,0,0,0,1,1,1,1

A	B	C
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1

1	1	0
1	1	1

Alternative method: Count in binary from 0 to 7: 000, 001, 010, 011, 100, 101, 110, 111.

> **Exam tip:** If the exam gives you a partially completed truth table, check the input combinations carefully. They might not be in the standard order!

2.3 Two-input truth tables

With two inputs, there are 4 possible combinations. Let's look at each operator:

A AND B — output 1 only when both are 1:

A	B	A AND B
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

Notice: Only the last row (both inputs = 1) produces output 1.

A OR B — output 1 when at least one is 1:

A	B	A OR B
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

Notice: Only the first row (both inputs = 0) produces output 0.

2.4 Three-input truth tables

With three inputs, there are 8 possible combinations. This is the maximum required at GCSE.

A AND B AND C — all three must be 1:

A	B	C	A AND B AND C
0	0	0	**0**
0	0	1	**0**
0	1	0	**0**

0	1	1	**0**
1	0	0	**0**
1	0	1	**0**
1	1	0	**0**
1	1	1	**1**

For A AND B AND C, only the row where all three inputs are 1 produces output 1. Seven rows produce 0, one row produces 1.

2.5 Working with intermediate columns

For complex expressions, add intermediate columns to break down the calculation. This makes it easier to track your working and helps you gain method marks.

Example: (A AND B) OR C

A	B	C	A AND B	(A AND B) OR C
0	0	0	0	**0**
0	0	1	0	**1**
0	1	0	0	**0**
0	1	1	0	**1**
1	0	0	0	**0**
1	0	1	0	**1**
1	1	0	1	**1**
1	1	1	1	**1**

The intermediate column "A AND B" helps calculate the final output step by step.

Worked Example 5 — Completing a two-input truth table

Question: Complete the truth table for A OR B. [2 marks]

Model answer:

A	B	A OR B
0	0	**0**
0	1	**1**
1	0	**1**

1	1	**1**

(1) for correct first two outputs, **(1)** for correct last two outputs.

Exam tip: For OR, remember: the output is 0 only when ALL inputs are 0. Otherwise it's 1.

Worked Example 6 — Truth table with NOT

Question: Complete the truth table for NOT A AND B. [2 marks]

Model answer:

A	B	NOT A	NOT A AND B
0	0	1	**0**
0	1	1	**1**
1	0	0	**0**
1	1	0	**0**

Method: First calculate NOT A for each row, then AND the result with B.

Exam tip: Adding an intermediate column for NOT A makes it much easier to get the final answer correct.

Worked Example 7 — Three-input truth table

Question: Complete the truth table for A OR B OR C. [3 marks]

Model answer:

A	B	C	A OR B OR C
0	0	0	**0**
0	0	1	**1**
0	1	0	**1**
0	1	1	**1**

1	0	0	**1**
1	0	1	**1**
1	1	0	**1**
1	1	1	**1**

Only the first row (000) outputs 0. All other rows output 1.

Exam tip: For multiple OR operations, the output is 0 only when ALL inputs are 0.

Worked Example 8 – Complex three-input expression

Question: Complete the truth table for (A AND B) OR (NOT C). [4 marks]

Model answer:

A	B	C	A AND B	NOT C	(A AND B) OR (NOT C)
0	0	0	0	1	**1**
0	0	1	0	0	**0**
0	1	0	0	1	**1**
0	1	1	0	0	**0**
1	0	0	0	1	**1**
1	0	1	0	0	**0**
1	1	0	1	1	**1**
1	1	1	1	0	**1**

Method: Calculate intermediate columns first, then combine with OR.

Exam tip: For 4-mark questions, use intermediate columns and check each row carefully. You can earn partial marks for correct intermediate values even if the final output is wrong.

3. Applying logic to solve problems

Boolean logic is used throughout programming to control the flow of programs. Understanding how to apply logical operators helps you write correct conditions and debug faulty code. Every selection statement, every loop condition, and every validation check uses Boolean logic.

> **Connection to programming:** This section links directly to Topic 6 (Problem solving with programming). The logical operators AND, OR, and NOT are used in Python as `and`, `or`, and `not` (lowercase).

3.1 Selection statements

The `if` statement uses Boolean conditions to decide which code to execute. The condition after `if` must evaluate to either TRUE or FALSE.

```
# AND example: both conditions must be true if age >= 18 and hasTicket ==
True: print("Entry allowed") # OR example: either condition can be true if
day == "Saturday" or day == "Sunday": print("It's the weekend!") # NOT
example: condition must be false if not loggedIn: print("Please log in")
```

3.2 Validation examples

Validation often combines multiple conditions to check if data meets specific requirements:

```
# Range check: value must be within range # Both conditions must be true
(AND) if score >= 0 and score <= 100: print("Valid score") else:
print("Score must be 0-100") # Multiple invalid inputs (OR) # Either
condition makes it invalid if password == "" or len(password) < 8:
print("Invalid password") else: print("Password accepted")
```

3.3 Loop conditions

Loops use Boolean conditions to decide when to continue or stop. The loop body executes as long as the condition is TRUE:

```
# Continue while BOTH conditions are true attempts = 0 success = False
while attempts < 3 and not success: password = input("Enter password: ")
if password == correctPassword: success = True attempts = attempts + 1 #
Stop when EITHER condition is met # (3 attempts used OR correct password
entered)
```

The loop stops when `attempts < 3` becomes FALSE (3 or more attempts) OR when `not success` becomes FALSE (success is TRUE).

3.4 Common logic patterns

These patterns appear frequently in exam questions and real programs:

Pattern 1: Range check

`value >= min AND value <= max`

True only when value is within the range [min, max]

Example: `age >= 18 and age <= 65`

Pattern 2: Either/or check

`option == "A" OR option == "B"`

True when either option is selected

Example: `colour == "red" or colour == "blue"`

Pattern 3: Exclusion check

`NOT (value == excluded)`

True when value is NOT the excluded value

Example: `not (answer == "quit")`

Pattern 4: Multiple requirements

`cond1 AND cond2 AND cond3`

True only when ALL conditions are met

Example: `hasID and age >= 18 and paid`

3.5 Debugging logic errors

Logic errors are mistakes in the reasoning of your code. They don't cause the program to crash, but they produce wrong results. Common logic errors to watch for:

- **Using AND instead of OR:** Makes condition too restrictive — fewer cases match
- **Using OR instead of AND:** Makes condition too permissive — too many cases match
- **Forgetting NOT:** Condition triggers at the wrong time (opposite of intended)
- **Wrong brackets:** Changes order of evaluation, giving unexpected results
- **Wrong relational operator:** Using > instead of >= misses boundary values

Debugging tip: Create a truth table for your condition and check it produces the expected output for each combination of inputs. Test boundary cases (e.g., exactly 18 for age >= 18).

Worked Example 9 — Writing a Boolean condition

Question: Write a Boolean condition that is TRUE when a student has scored 50 or more AND has attended at least 80% of classes. Use variables `score` and `attendance`. [2 marks]

Model answer:

`score >= 50 AND attendance >= 80` **(1)** for each correct sub-condition

Or in Python: `score >= 50 and attendance >= 80`

Exam tip: Read the question carefully. "50 or more" means >= 50, not > 50. "At least 80%" means >= 80.

Worked Example 10 – Identifying logic errors

Question: A program should allow access if the user is an admin OR has permission level 3 or higher. The code is: `if role == "admin" AND permissionLevel >= 3:` Explain why this code is incorrect. [2 marks]

Model answer:
The code uses AND instead of OR **(1)**.
This means BOTH conditions must be true, but the requirement states either condition should grant access **(1)**.
Correct code: `if role == "admin" or permissionLevel >= 3:`

Exam tip: When the question says "either...or", you need the OR operator. When it says "both...and", you need the AND operator.

Worked Example 11 – Evaluating code with logic

Question: What is the output of this code when x = 5 and y = 10? [2 marks]

```
if (x > 3) and (y < 8): print("A") elif (x > 3) or (y < 8): print("B")
else: print("C")
```

Model answer:
Step 1: First condition: (5 > 3) AND (10 < 8) = TRUE AND FALSE = FALSE **(1)**
Step 2: Second condition: (5 > 3) OR (10 < 8) = TRUE OR FALSE = TRUE
Output: B **(1)**

Exam tip: Evaluate each sub-condition separately, then combine. Show your working!

Worked Example 12 – Real-world application

Question: A theme park ride has these requirements: riders must be at least 140cm tall AND (aged 12 or over OR accompanied by an adult). Complete the truth table for the ride access logic. [4 marks]

Let: T = tallEnough, A = ageOK (12+), C = accompanied
Expression: T AND (A OR C)

T	A	C	A OR C	T AND (A OR C)

0	0	0	0	**0**
0	0	1	1	**0**
0	1	0	1	**0**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	1	1	**1**
1	1	0	1	**1**
1	1	1	1	**1**

Access is granted only when tall enough AND either old enough OR accompanied.

Exam tip: Real-world problems often combine AND and OR. Identify the key conditions and translate them into logical expressions before building the truth table.

Worked examples summary

The table below summarises the 12 worked examples covered in this topic.

#	Topic	Question type	Marks	Key skill
1	AND operator	Identify output	1	Both inputs must be TRUE
2	OR operator	Identify output	1	At least one input TRUE
3	NOT operator	Apply NOT	1	Invert the value
4	Combined expression	Evaluate expression	2	Order of operations
5	Two-input table	Complete table	2	A OR B
6	NOT in table	Complete table	2	NOT A AND B
7	Three-input table	Complete table	3	A OR B OR C
8	Complex expression	Complete table	4	Intermediate columns
9	Writing conditions	Write expression	2	Translate requirements
10	Logic errors	Identify error	2	AND vs OR

#	Topic	Question type	Marks	Key skill
11	Code evaluation	Determine output	2	Trace through code
12	Real-world problem	Complete table	4	Apply logic to scenario

Quick reference

Key terms and definitions

Term	Definition (mark-scheme style)
Boolean	A data type that can only have two values: TRUE or FALSE (1 or 0)
Logical operator	An operator that takes Boolean input(s) and produces a Boolean output
AND	Returns TRUE only when all inputs are TRUE
OR	Returns TRUE when at least one input is TRUE
NOT	Returns the opposite of the input (inverts TRUE to FALSE and vice versa)
Truth table	A table showing all possible input combinations and their corresponding outputs
Condition	A Boolean expression that evaluates to TRUE or FALSE

Operator summary

Operator	Inputs	Output is TRUE when...	Output is FALSE when...
AND	2 or more	ALL inputs are TRUE	ANY input is FALSE
OR	2 or more	AT LEAST ONE input is TRUE	ALL inputs are FALSE
NOT	1	Input is FALSE	Input is TRUE

Truth table quick patterns

Expression	Number of 1s in output	Pattern
A AND B (2 inputs)	1 out of 4	Only 1,1 row is TRUE
A OR B (2 inputs)	3 out of 4	Only 0,0 row is FALSE
A AND B AND C (3 inputs)	1 out of 8	Only 1,1,1 row is TRUE
A OR B OR C (3 inputs)	7 out of 8	Only 0,0,0 row is FALSE

Typical pitfalls

Pitfall 1 — Confusing AND and OR.
Wrong: Using AND when "either" condition should work.
Right: "Either A or B" uses OR. "Both A and B" uses AND. Read the question carefully.

Pitfall 2 — Wrong number of rows.
Wrong: Creating 4 rows for a 3-input truth table.
Right: Use 2^n rows: 1 input = 2 rows, 2 inputs = 4 rows, 3 inputs = 8 rows.

Pitfall 3 — Missing input combinations.
Wrong: Skipping combinations like 0,1,0 in a 3-input table.
Right: Use binary counting order: 000, 001, 010, 011, 100, 101, 110, 111.

Pitfall 4 — Forgetting order of operations.
Wrong: Evaluating A OR B AND C as (A OR B) AND C.
Right: AND is evaluated before OR: A OR (B AND C). Use brackets to be clear.

Pitfall 5 — NOT applies to one input only.
Wrong: NOT (A AND B) means NOT A AND NOT B.
Right: NOT (A AND B) inverts the entire result of A AND B, not each input separately.

Pitfall 6 — Using 1/0 and TRUE/FALSE inconsistently.
Wrong: Mixing "1" and "TRUE" in the same truth table.
Right: Be consistent throughout. Use either 1/0 or TRUE/FALSE, not both.

Pitfall 7 — Not showing intermediate columns.
Wrong: Jumping straight to the final answer for complex expressions.
Right: Add intermediate columns for partial calculations — this earns method marks.

Pitfall 8 — Misreading >= and >.
Wrong: Treating "at least 50" as > 50.
Right: "At least 50" means >= 50 (50 or more). "More than 50" means > 50.

Pitfall 9 — Forgetting Python syntax.
Wrong: Writing `AND` or `OR` in Python code.
Right: Python uses lowercase: `and`, `or`, `not`.

Pitfall 10 — Not checking all rows.
Wrong: Only checking a few rows and assuming the pattern continues.
Right: Complete every row of the truth table. Each row must be evaluated individually.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.